
DENNIS GUNAWAN



UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

IF351 DATABASE SYSTEMS

03 DATA MODELING USING THE ENTITY-RELATIONSHIP MODEL



REVIEW:INTRODUCTION TO DATABASE SYSTEM

- Database
- Database Management System (DBMS)
- Characteristics of the Database Approach
- Actors on the Scene
- Advantages of Using the DBMS Approach
- When Not to Use a DBMS
- Data Model
- Three-Schema Architecture
- Data Independence
- DBMS Languages
- Database System Environment
- Architectures for DBMSs

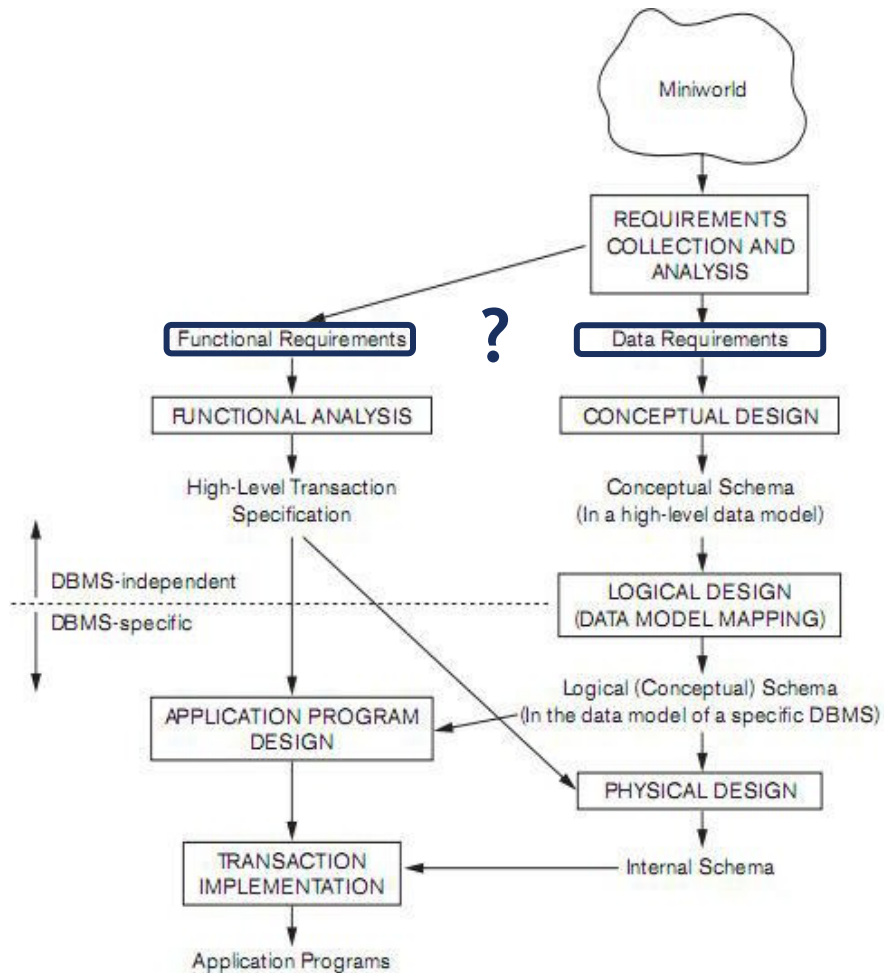
COURSE SUB LEARNING OUTCOMES (SUB-CLO)

- Sub-CLO 2
 - Students are able to use Entity-Relationship model for data modeling (C3)

OUTLINE

- Database Design
- Entities
- Attributes
- Relationships
- ER Diagrams Notation

DATABASE DESIGN



Functional Requirements
VS
Data Requirements

REQUIREMENTS COLLECTION AND ANALYSIS

- Database designers interview prospective database users to understand and document their data requirements
- The result of this step is a concisely written set of users' requirements
- **Functional requirements:** user-defined operations or transactions that will be applied to the database (what functions do users need)
 - I want a shopping cart function
- **Data requirements:** what data do users need
 - For the shopping cart function, users' account and product items are needed

CONCEPTUAL DESIGN

- Create a conceptual schema for the database, using a high-level conceptual data model
- A concise description of the data requirements of the users
 - Includes detailed descriptions of the entity types, relationships, and constraints
- Do not include implementation details
 - Usually easier to understand
 - Can be used to communicate with nontechnical users
- Can also be used as a reference to ensure that all users' data requirements are met and that the requirements do not conflict

LOGICAL DESIGN

- The actual implementation of the database, using a commercial DBMS
- Most current commercial DBMSs use an implementation / relational data model
- Its result is a database schema in the implementation data model of the DBMS

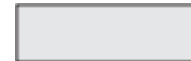
PHYSICAL DESIGN

- Internal storage structures, indexes, access paths, and file organizations for the database files are specified

ENTITIES

A thing in the real world
with an independent existence

- An object with a physical existence
 - A particular person, car, house, or employee
- An object with a conceptual existence
 - A company, a job, or a university course
- Notation



ATTRIBUTES

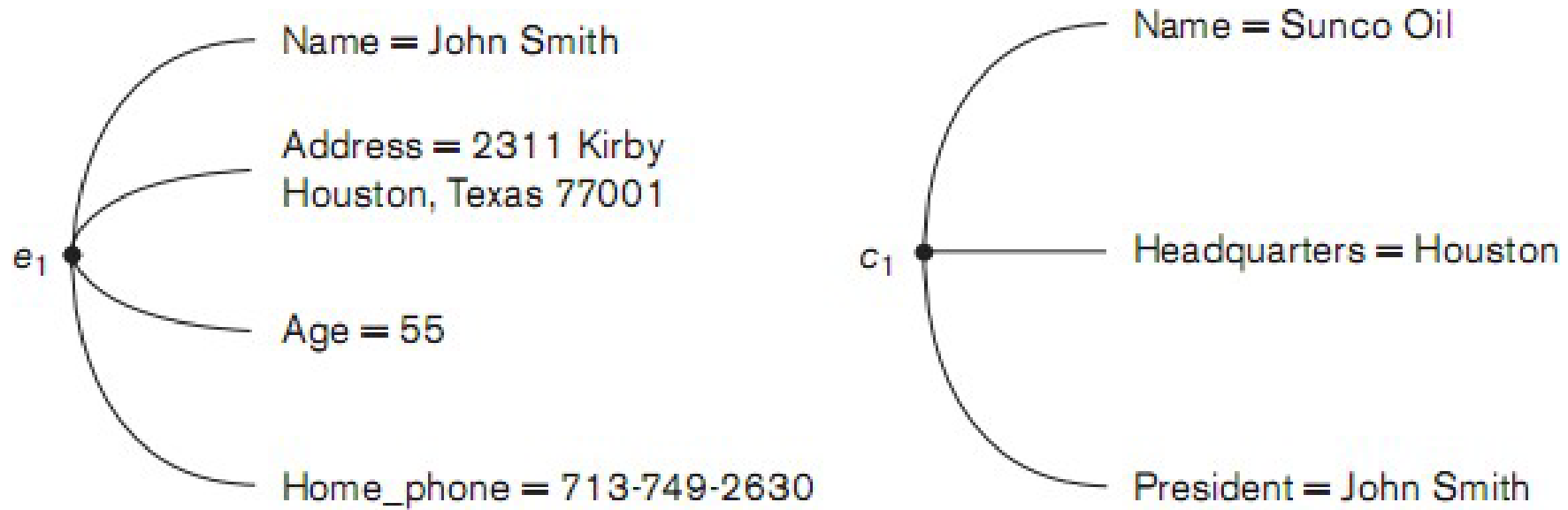
Each entity has attributes:
The particular properties that describe it

- For example, an EMPLOYEE entity may be described by the employee's name, age, address, salary, and job
- The attribute values that describe each entity become a major part of the data stored in the database

- Notation



ATTRIBUTES



- Attributes of EMPLOYEE entity e_1 ?
- Attributes of COMPANY entity c_1 ?

TYPES OF ATTRIBUTES: SIMPLE (ATOMIC) VS COMPOSITE ATTRIBUTES

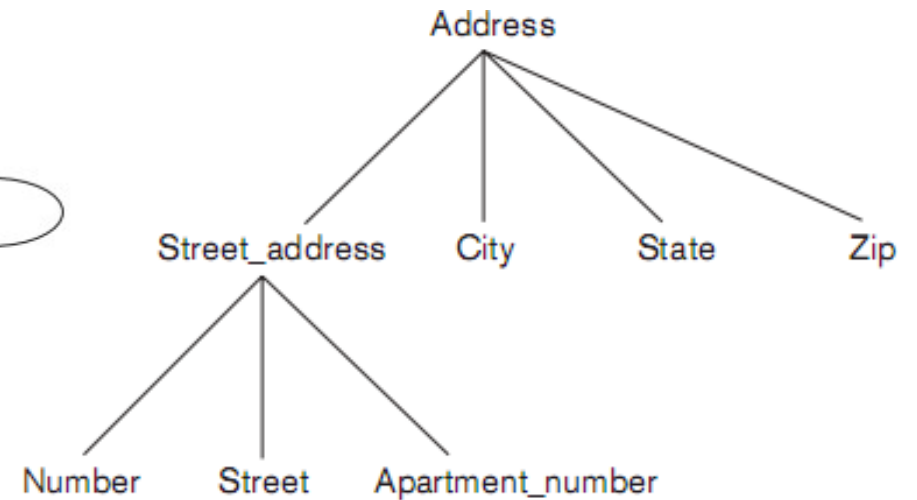
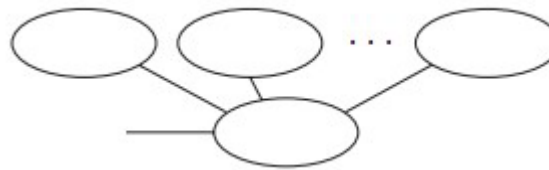
Simple (Atomic) Attributes

- Attributes that are not divisible

Composite Attributes

- Can be divided into smaller subparts, which represent more basic attributes with independent meanings

- Notation



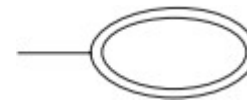
TYPES OF ATTRIBUTES: SINGLE-VALUED VS MULTIVALUED ATTRIBUTES

Single-Valued Attributes

- Attributes that have a single value for a particular entity

Multivalued Attributes

- Attributes that have a set of values for a particular entity
- Notation



TYPES OF ATTRIBUTES: STORED VS DERIVED ATTRIBUTES

- For a particular PERSON entity, the value of *Age* can be determined from the current date and the value of that person's *Birth_date*
- The *Age* attribute is called a **derived attribute** and is said to be derivable from the *Birth_date* attribute, which is called a **stored attribute**
- Notation



TYPES OF ATTRIBUTES: COMPLEX ATTRIBUTES

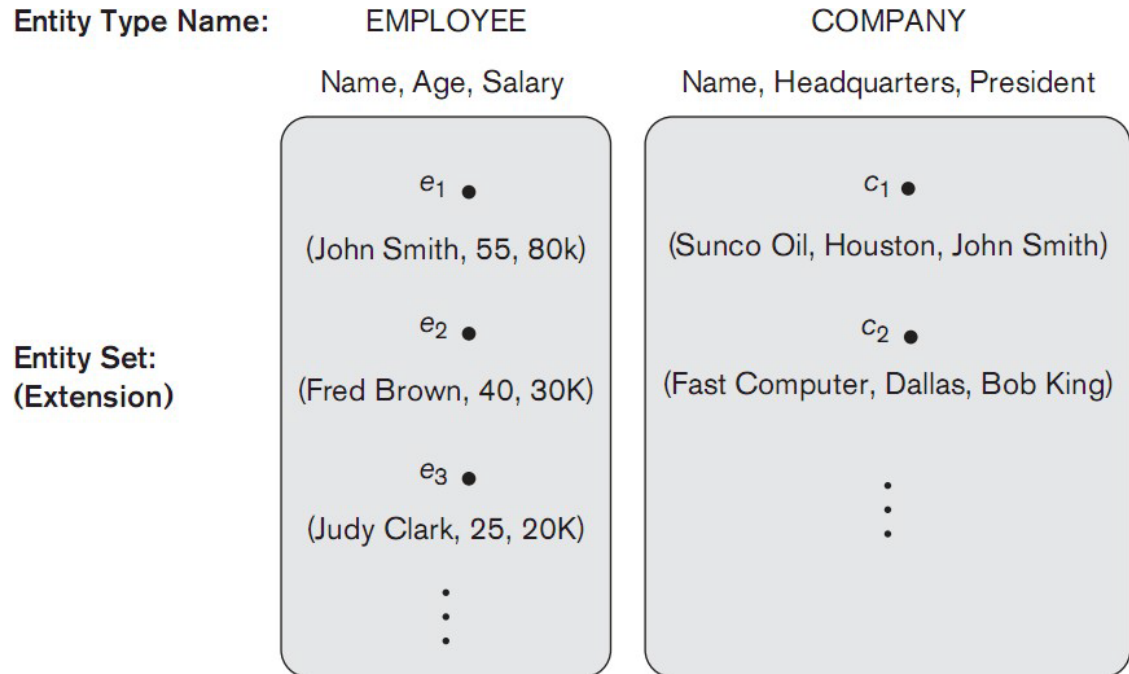
Attributes which are comprised of
multivalued and **composite** attributes

- A person can have more than one residence and each residence can have a single address and multiple phones
 - {Residence({Phone(Area_code,Phone_number)},
Address(Street_address(Number,Street,Apartment_number),
City, State, Zip))}

NULL VALUES

- In some cases, a particular entity may **not** have an **applicable** value for an attribute
 - A *College_degrees* attribute applies only to persons with college degrees
 - A person with no college degree would have NULL for *College_degrees*
- NULL can also be used if we do **not know** the value of an attribute for a particular entity
 - For example, if we do not know the phone number of Richard
- The meaning of the former type of NULL is **not applicable**, whereas the meaning of the latter is **unknown**

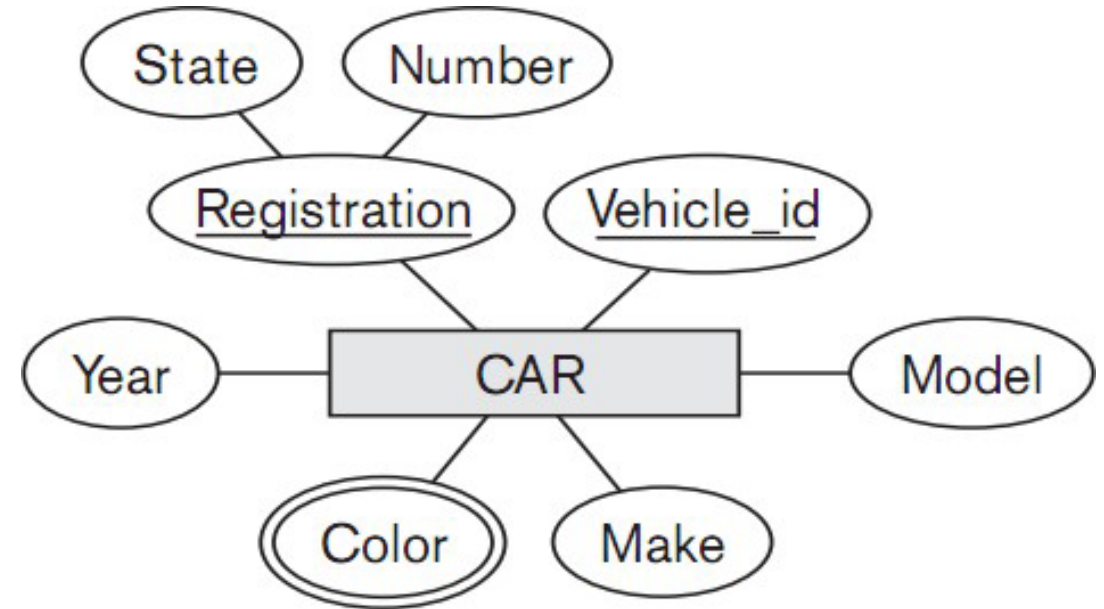
ENTITY TYPES & ENTITY SETS



- **Entity type** defines a collection / set of entities that have the same attributes
- Each entity type in the database is described by its name and attributes
- **Entity set:** the collection of all entities of a particular entity type in the database at any point in time

ENTITY TYPES & ENTITY SETS

- An entity type is represented in ER diagrams as a rectangular box enclosing the entity type name
- Attribute names are enclosed in ovals and are attached to their entity type by straight lines
- Composite attributes are attached to their component attributes by straight lines
- Multivalued attributes are displayed in double ovals



KEY ATTRIBUTES OF AN ENTITY TYPE

- An important constraint on the entities of an entity type is the **key** or **uniqueness constraint** on attributes
- An entity type usually has an attribute whose values are distinct for each individual entity in the entity set
- Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely
- For the STUDENT entity type, a typical key attribute is *Student_id* because no two students are allowed to have the same student ID

KEY ATTRIBUTES OF AN ENTITY TYPE

- Sometimes several attributes together form a key, meaning that the combination of the attribute values must be distinct for each entity
- In ER diagrammatic notation, each key attribute has its name **underlined** inside the oval

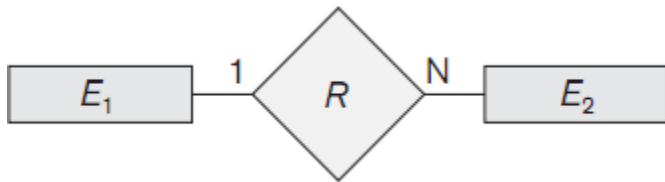


VALUE SETS (DOMAINS) OF ATTRIBUTES

- Each simple attribute of an entity type is associated with a **value set** (or **domain** of values), which specifies the set of values that may be assigned to that attribute for each individual entity
- If the range of ages allowed for employees is between 16 and 70, we can specify the value set of the *Age* attribute of EMPLOYEE to be the set of integer numbers between 16 and 70
- Value sets are not displayed in ER diagrams
- Value sets are typically specified using the data types available in most programming languages, such as integer, string, Boolean, float, date, time, and so on

RELATIONSHIPS

- A relationship type among n entity types defines a set of associations among entities from these entity types
- In ER diagrams, relationship types are displayed as diamond-shaped boxes, which are connected by straight lines to the rectangular boxes representing the participating entity types
- The relationship name is displayed in the diamond-shaped box

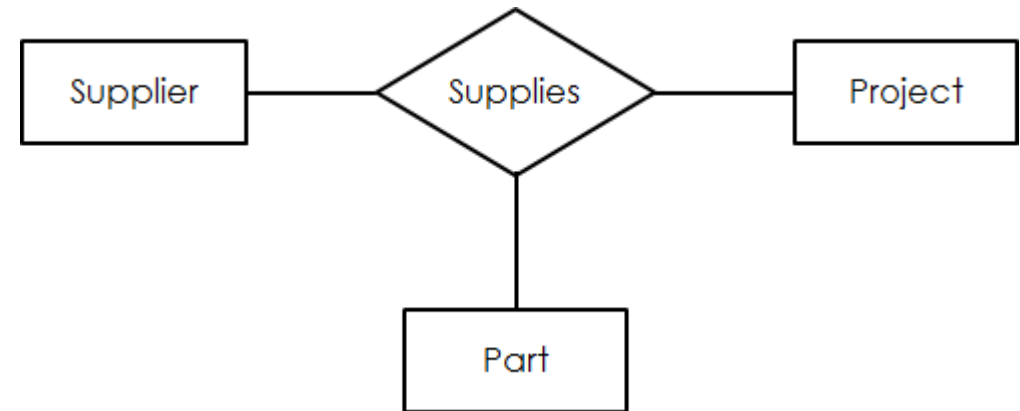
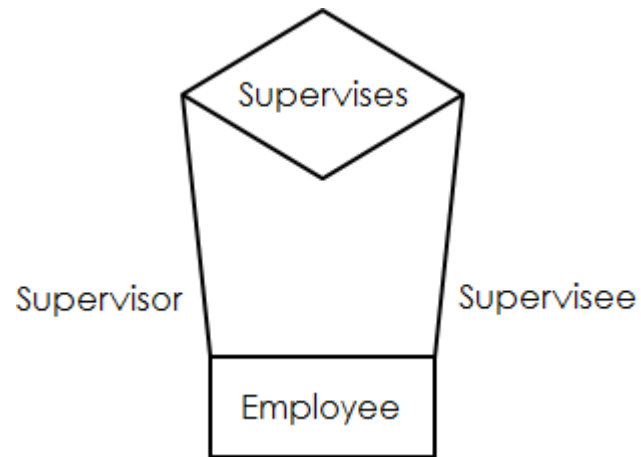


DEGREE OF A RELATIONSHIP TYPE

The number of
participating entity types

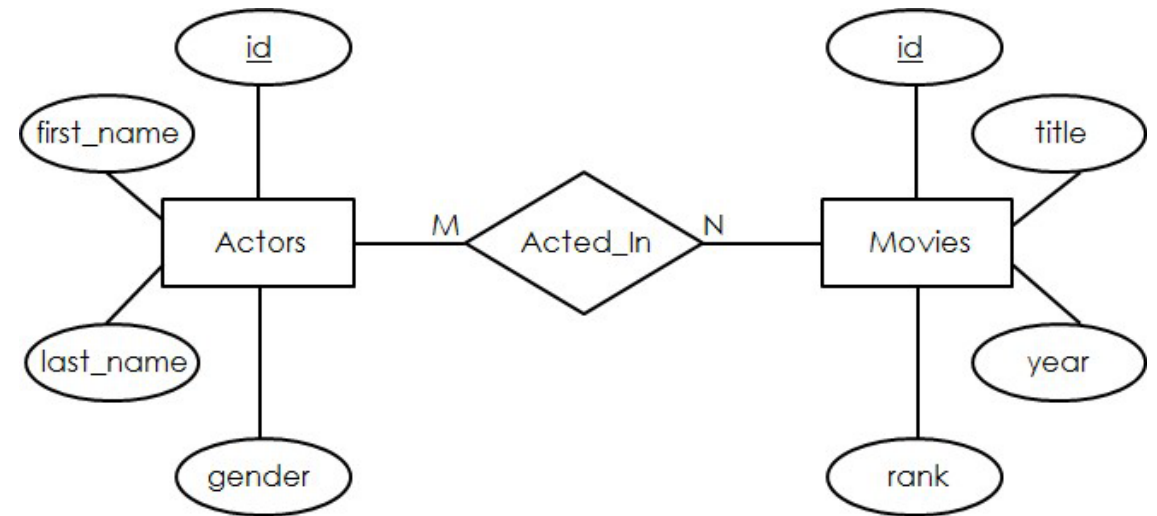
- **Unary (recursive)**: a relationship type of degree one
 - The same entity type participates more than once in a relationship type in different roles
- **Binary**: a relationship type of degree two
- **Ternary**: a relationship type of degree three
- **N-ary**: a relationship type of degree N where $N > 3$

DEGREE OF A RELATIONSHIP TYPE



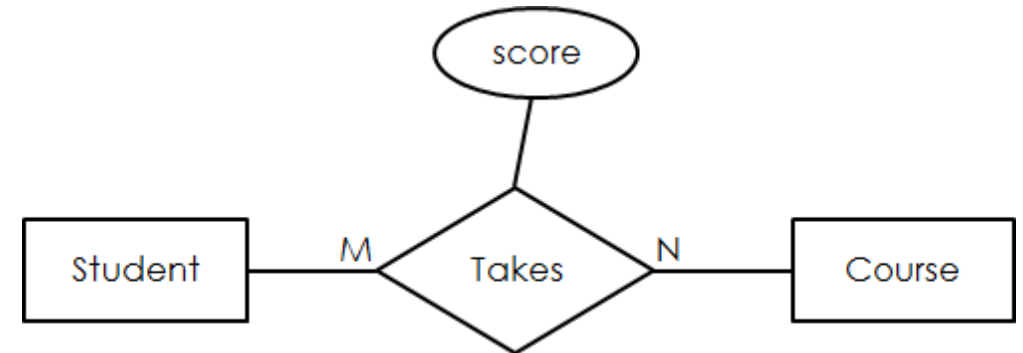
CARDINALITY RATIOS FOR BINARY RELATIONSHIPS

- Cardinality ratio for most unary relationship is 1:N
- Cardinality ratio for most ternary relationship is N:N:N
- Cardinality ratio for a binary relationship specifies the maximum number of relationship instances that an entity can participate in
- The possible cardinality ratios for binary relationship types are 1:1, 1:N, N:1, and M:N



ATTRIBUTES OF RELATIONSHIP TYPES

- Relationship types can also have attributes
- Notice that attributes of 1:1 or 1:N relationship types can be migrated to one of the participating entity types
- For a 1:N relationship type, a relationship attribute can be migrated only to the entity type on the N-side of the relationship
- For M:N relationship types, some attributes may be determined by the combination of participating entities in a relationship instance, not by any single entity
 - Such attributes must be specified as relationship attributes



WEAK ENTITY TYPES

Weak Entity Types

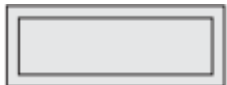
Entity types that do not have key attributes of their own

Strong Entity Types

Regular entity types that do have a key attribute

WEAK ENTITY TYPES

- A weak entity type always has a total participation constraint (existence dependency) with respect to its identifying relationship because a weak entity cannot be identified without an owner entity
- Not every existence dependency results in a weak entity type
 - For example, a DRIVER_LICENSE entity cannot exist unless it is related to a PERSON entity, even though it has its own key (*License_number*) and hence is not a weak entity
- A weak entity type normally has a **partial key**, which is the set of attributes that can uniquely identify weak entities that are related to the same owner entity
- Notation

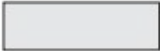
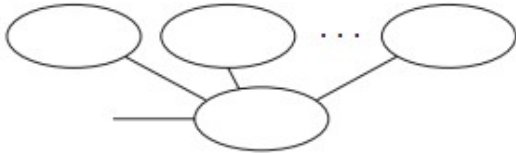
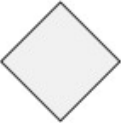
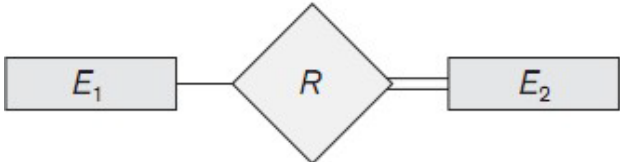
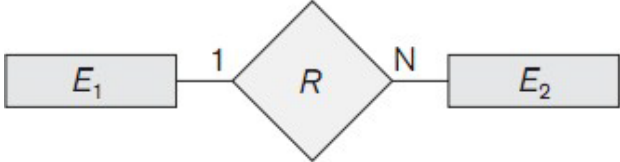
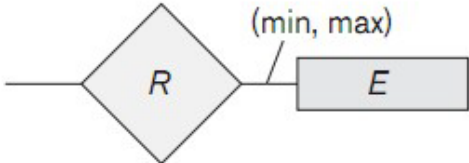


Weak Entity



Identifying Relationship

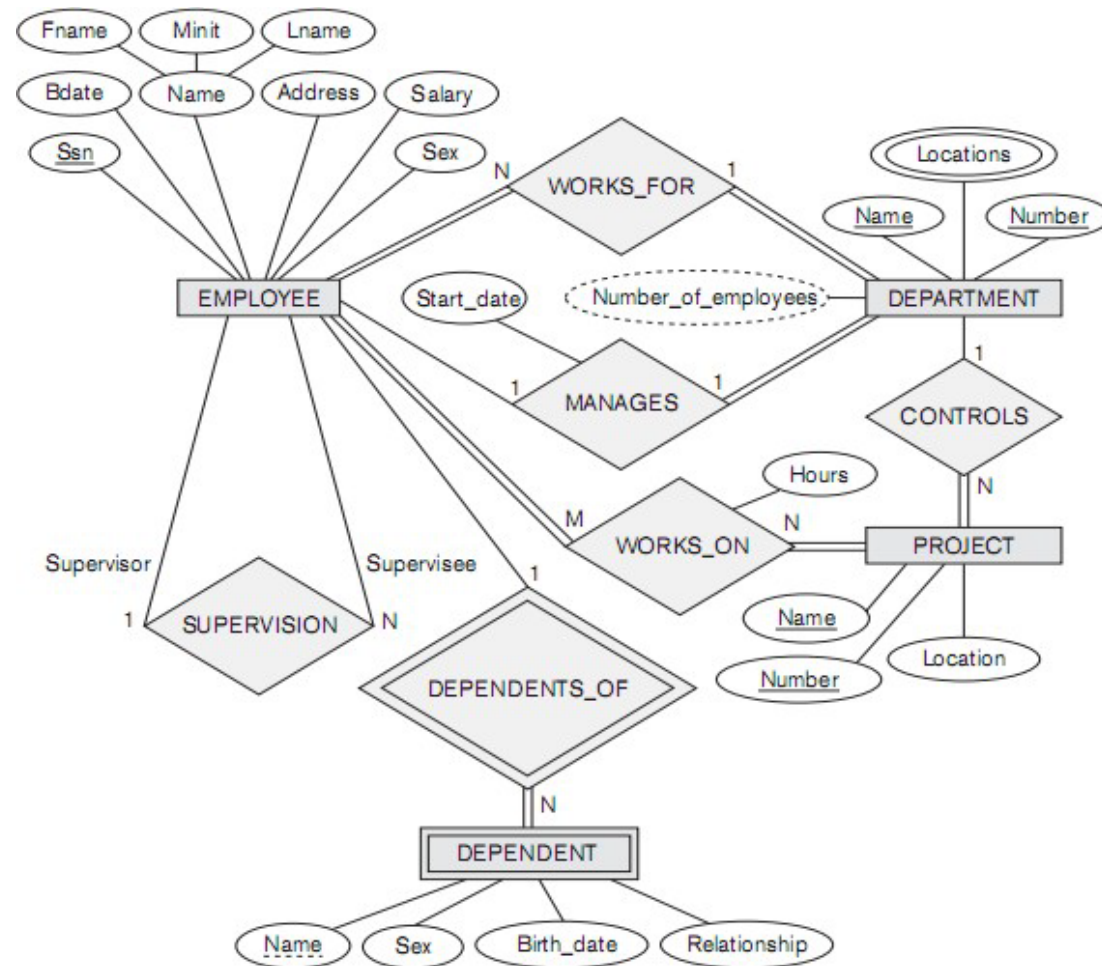
ER DIAGRAMS NOTATION

Symbol	Meaning		
	Entity		Composite Attribute
	Weak Entity		Derived Attribute
	Relationship		Total Participation of E_2 in R
	Identifying Relationship		Cardinality Ratio 1: N for $E_1:E_2$ in R
	Attribute		Structural Constraint (min, max) on Participation of E in R
	Key Attribute		
	Multivalued Attribute		

EXAMPLE

- The company is organized into departments. Each department has a unique name, a unique number, and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations.
- A department controls a number of projects, each of which has a unique name, a unique number, and a single location.
- We store each employee's name, social security number, address, salary, sex, and birth date. An employee is assigned to one department, but may work on several projects, which are not necessarily controlled by the same department. We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee.
- We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's first name, sex, birth date, and relationship to the employee.

EXAMPLE



REFERENCES

- Elmasri, Ramez and Shamkant B. Navathe (2016), Fundamentals of Database Systems, 7th edition, Addison Wesley.
- Rick F.van der Lans (2007), SQL for MySQL Developers, Addison Wesley.
- Connolly, Thomas and Carolyn Begg (2015), Database Systems A Practical Approach to Design, Implementation, and Management, 6th edition, Pearson Education, ISBN 13: 978-1-292-06118-4, Essex, England.

NEXT WEEK:

THE RELATIONAL DATA MODEL, RELATIONAL DATABASE CONSTRAINTS, AND RELATIONAL DATABASE DESIGN

- Relational Data Model
- Relational Database Constraints
- ER-to-Relational Mapping Algorithm

```
SELECT *  
FROM WORLD  
WHERE SOMEONE  
LIKE '%YOU%';
```

```
EMPTY SET (0.00 SEC)
```

Vision

To become an **outstanding** undergraduate Computer Science program that produces **international-minded** graduates who are **competent** in software engineering and have **entrepreneurial spirit** and **noble character**.



Mission

1. To conduct studies with the best technology and curriculum, supported by professional lecturer
2. To conduct research in Informatics to promote science and technology
3. To deliver science-and-technology-based society services to implement science and technology